

REKÜRSİF FONKSİYONLAR – Zaman Karmaşıklığı Analizi

Çalışma Kağıdı | Yineleme Bağıntısı (Recurrence Relation)

Temel Yöntem: Rekürsif bir fonksiyonun zaman karmaşıklığını bulmak için önce yineleme bağıntısı $T(n)$ yazılır, ardından *ardışık yerine koyma* (*substitution / unrolling*) yöntemiyle kapalı form elde edilir.

Genel Strateji: $T(n)$ ifadesinde $T(n-1)$ veya $T(n/2)$ gibi daha küçük argümanları adım adım açın. Her adımda ortaya çıkan deseni (i 'nin fonksiyonu) fark edin; taban durumuna ulaşana kadar devam edin. Sonra i 'yi n 'in cinsinden yazıp kapalı formu elde edin.

1 1

Doğrusal

Azalma – $T(n) = T(n-1) + c$

Fonksiyon her çağrıda problemi bir küçülterek ilerler. Bu en temel rekürsif örüntüdür; toplam maliyet $\Theta(n)$ olur.

Örnek 1.1 – Tek özyinelemeli çağrı, sabit iş

```
void f(int n) {
    if (n == 0) return;    // T(0) = c
    f(n-1);                // T(n-1)
}
```

Yineleme bağıntısı:

$$T(n) = T(n-1) + c, \quad T(0) = c$$

Açma (unrolling):

$$\begin{aligned} T(n) &= T(n-1) + c \\ T(n-1) &= T(n-2) + c \\ T(n-2) &= T(n-3) + c \\ &\vdots \\ T(1) &= T(0) + c \end{aligned}$$

n satır toplandığında $T(0) = c$ ile:

$$T(n) = (n+1) \cdot c = \boxed{\Theta(n)}$$

Pratik Soru 1.1

Aşağıdaki fonksiyonun $T(n)$ bağıntısını yazın ve çözün:

```
void g(int n) {
    if (n == 0) return;
    printf("*"); // sabit iş
    g(n-1);
}
```

Örnek 1.2 – Özyinelemeli çağrı + doğrusal döngü

Fonksiyon hem kendini çağırır hem de n uzunluklu bir döngü çalıştırır. Döngü her seviyede ayrı bir maliyet ekler; toplam $\Theta(n^2)$ olur.

Örnek 1.2

```
void f(int n) {
    if (n == 0) return; // c_1
    for (int i = 0; i < n; i++); // c_2 * n
    f(n-1);
}
```

Yineleme bağıntısı:

$$T(n) = T(n-1) + c_2n + c_1, \quad T(0) = c_1$$

Açma:

$$\begin{aligned} T(n) &= T(n-1) + c_2n + c_1 \\ T(n-1) &= T(n-2) + c_2(n-1) + c_1 \\ T(n-2) &= T(n-3) + c_2(n-2) + c_1 \\ &\vdots \end{aligned}$$

n adım sonra $T(0) = c_1$ ile:

$$T(n) = c_2 \sum_{k=1}^n k + (n+1)c_1 = c_2 \cdot \frac{n(n+1)}{2} + (n+1)c_1 = \boxed{\Theta(n^2)}$$

Pratik Soru 1.2

Aşağıdaki bağıntıyı çözün ve büyük-O karmaşıklığını bulun:

$$T(n) = T(n-1) + 5n, \quad T(0) = 1$$

2 1**Logaritmik****Azalma** – $T(n) = T(n/2) + c$

Fonksiyon her çağrıda n 'i yarıya böler. Kaç kez bölünebilir? $\log_2 n$ kez! Bu nedenle toplam maliyet $\Theta(\log n)$ olur.

Örnek 2.1 – İkiye bölme, sabit iş

```
void f(int n) {
    if (n == 1) return;    // T(1) = c
    f(n/2);
}
```

Yineleme bağıntısı:

$$T(n) = T(n/2) + c, \quad T(1) = c$$

Açma:

$$T(n) = T(n/2^i) + i \cdot c$$

$n/2^i = 1 \Rightarrow i = \log_2 n$ olduğunda taban durumuna ulaşılır:

$$T(n) = T(1) + c \log n = c(\log n + 1) = \boxed{\Theta(\log n)}$$

Pratik Soru 2.1

$T(n) = T(n/3) + c$, $T(1) = c$ için kapalı formu bulun. _____

Örnek 2.2 – İkiye bölme + doğrusal döngü

Yarı-yarıya bölme ile birlikte n uzunluklu döngü eklenince toplam $\Theta(n \log n)$ elde edilir.

Örnek 2.2

```
void f(int n) {
    if (n == 0) return;    // c_1
    for (int i = 0; i < n; i++); // c_2 * n
    f(n/2);
}
```

Yineleme bağıntısı:

$$T(n) = T(n/2) + c_2n + c_1, \quad T(0) = c_2$$

Açma (i adım sonra):

$$T(n) = T\left(\frac{n}{2^i}\right) + c_2n\left(\frac{1}{2^0} + \frac{1}{2^1} + \cdots + \frac{1}{2^{i-1}}\right) + i c_1$$

$\log n$ adım sonra, geometrik seri $\sum_{k=0}^{\log n - 1} (1/2)^k \rightarrow 2$ ile:

$$T(n) = c_1 + c_2(2 + \log n) + c_1 \log n = c_2(2 + \log n) + c_1(\log n + 1) = \boxed{\Theta(n \log n)}$$

Not: Geometrik seri kısmi toplamı $\sum_{k=0}^{m-1} \frac{1}{2^k} = 2 - \frac{1}{2^{m-1}} < 2$

Pratik Soru 2.2

$T(n) = T(n/2) + n$, $T(1) = 1$ bağıntısını çözün. _____

3 1

İki

Özyinelemeli Çağrı – $T(n) = 2T(n-1) + c$

Fonksiyon kendini iki kez çağırdığında çağrı ağacı üstel büyür. Her seviyede çağrı sayısı iki katına çıktığından toplam maliyet $\Theta(2^n)$ olur.

Örnek 3 – İki kez azaltma

```
void f(int n) {
    if (n == 0) return; // T(0) = c
    f(n-1);
    f(n-1);
}
```

Yineleme bağıntısı:

$$T(n) = 2T(n-1) + c, \quad T(0) = c, \quad T(1) = 3c$$

Açma:

$$\begin{aligned}
T(n) &= 2T(n-1) + c \\
&= 2[2T(n-2) + c] + c = 2^2T(n-2) + 2c + c \\
&= 2^2[2T(n-3) + c] + 2c + c = 2^3T(n-3) + 2^2c + 2c + c \\
&\vdots \\
&= 2^iT(n-i) + c(2^{i-1} + \dots + 2^0) = 2^iT(n-i) + c(2^i - 1)
\end{aligned}$$

$n - i = 0 \Rightarrow i = n$ olduğunda:

$$T(n) = 2^n \cdot T(0) + c(2^n - 1) = 2^n c + c(2^n - 1) = c(2^{n+1} - 1) = \boxed{\Theta(2^n)}$$

Pratik Soru 3

$T(n) = 2T(n-1) + 1$, $T(0) = 1$ bağıntısını çözün. _____

4 1

İki

Özyinelemeli Çağrı + Bölme – $T(n) = 2T(n/2) + c$

Merge Sort gibi algoritmalarda fonksiyon hem ikiye böler hem de her iki yarıyı özyinelemeli çağırır. Çağrı ağacı $\log n$ derinliğe sahip olmakla birlikte, her seviyede 2^k çağrı sabit iş yapar; toplam $\Theta(n)$ olur.

Örnek 4.1 – Böl ve fethet, sabit birleştirme

```
void f(int n) {
    if (n == 0) return;    // T(0) = c
    f(n/2);
    f(n/2);
}
```

Yineleme bağıntısı:

$$T(n) = 2T(n/2) + c, \quad T(0) = c, \quad T(1) = 3c$$

Açma:

$$\begin{aligned}
T(n) &= 2T(n/2) + c \\
T(n/2) &= 2T(n/4) + c \\
T(n) &= 2^2T(n/4) + 2c + c \\
T(n/4) &= 2T(n/8) + c \\
T(n) &= 2^3T(n/8) + 2^2c + 2c + c \\
&\vdots \\
T(n) &= 2^iT(n/2^i) + c(2^{i-1} + \dots + 1) = 2^iT(n/2^i) + c(2^i - 1)
\end{aligned}$$

$$n/2^i = 1 \Rightarrow i = \log n:$$

$$T(n) = n \cdot T(1) + c(n - 1) = 3cn + c(n - 1) = 4cn - c = \boxed{\Theta(n)}$$

Pratik Soru 4.1

$T(n) = 2T(n/2) + c$, $T(1) = c$ için kapalı formu bulun ve Θ karmaşıklığını yazın. _

Örnek 4.2 – Böl ve fethet + doğrusal birleştirme (Merge Sort)

Her seviyede n büyüklüğünde birleştirme (merge) işlemi yapılır ve $\log n$ derinlik olduğundan toplam maliyet $\Theta(n \log n)$ çıkar.

Örnek 4.2

```

void f(int n) {
    if (n == 0) return;           // c_1
    for (int i = 0; i < n; i++); // c_2 * n (merge adımı)
    f(n/2);
    f(n/2);
}

```

Yineleme bağıntısı:

$$T(n) = 2T(n/2) + c_2n + c_1, \quad T(0) = c_2$$

Açma (i adım):

$$T(n) = 2^iT(n/2^i) + c_2n \underbrace{(1 + 1 + \dots + 1)}_{i \text{ kez}} + c_1(2^i - 1)$$

$i = \log n$ olduğunda:

$$T(n) = n \cdot T(1) + c_2n \log n + c_1(n - 1) = \boxed{\Theta(n \log n)}$$

Ağaç görselleştirmesi:

$$\underbrace{n}_{\text{seviye 0}} \rightarrow \underbrace{n/2 + n/2}_{\text{seviye 1}} \rightarrow \underbrace{n/4 \times 4}_{\text{seviye 2}} \rightarrow \dots \quad (\log n \text{ seviye})$$

Her seviyede toplam iş: $n \Rightarrow n \cdot \log n$

Pratik Soru 4.2

Aşağıdaki bağıntıyı çözün:

$$T(n) = 2T(n/2) + n^2, \quad T(1) = 1$$

5 1 İki Özyinelemeli Çağrı + Azaltma – $T(n) = 2T(n-1) + c$

İki çağrıda $n-1$ ile gidilirse ağaç derinliği n , genişliği 2^n olur; üstel büyüme kaçınılmaz.

Örnek 5 – İkili azaltma (Fibonacci benzeri)

```
void f(int n) {
    if (n == 0) return; // c
    f(n-1);
    f(n-1);
}
```

Yineleme bağıntısı:

$$T(n) = 2T(n-1) + c, \quad T(0) = c$$

Kapalı form (Örnek 3'ten):

$$T(n) = c(2^{n+1} - 1) = \boxed{\Theta(2^n)}$$

Neden bu kadar büyük? Seviye k 'da 2^k çağrı yapılır; toplam çağrı sayısı:

$$\sum_{k=0}^n 2^k = 2^{n+1} - 1$$

Pratik Soru 5

$T(n) = 2T(n-1) + n$, $T(0) = 1$ bağıntısı hakkında ne söylenebilir? Büyük- O üst sınırını bulun. _____

6 1 Tablosu

Özet

Yineleme Bağıntısı	Kapalı Form	Θ	Örnek
$T(n) = T(n-1) + c$	$c(n+1)$	$\Theta(n)$	1.1
$T(n) = T(n-1) + cn$	$c \frac{n(n+1)}{2}$	$\Theta(n^2)$	1.2
$T(n) = T(n/2) + c$	$c(\log n + 1)$	$\Theta(\log n)$	2.1
$T(n) = T(n/2) + cn$	$c_2(2 + \log n) + c_1 \log n$	$\Theta(n \log n)$	2.2
$T(n) = 2T(n-1) + c$	$c(2^{n+1} - 1)$	$\Theta(2^n)$	3
$T(n) = 2T(n/2) + c$	$4cn - c$	$\Theta(n)$	4.1
$T(n) = 2T(n/2) + cn$	$cn \log n + \dots$	$\Theta(n \log n)$	4.2

Karar Ağacı:

- $T(n-1)$ ve tek çağrı $\Rightarrow$ en az $\Theta(n)$
- $T(n/2)$ ve tek çağrı $\Rightarrow$ en az $\Theta(\log n)$
- $2T(n-1) \Rightarrow \Theta(2^n)$ (üstel – kaçının!)
- $2T(n/2) \Rightarrow \Theta(n)$ (bölme+fethet verimi)
- Her durumda eklenen $f(n)$ terimi toplamı etkileyebilir

Ek Pratik – Karma Sorular

- S1.** $T(n) = T(n-2) + c$, $T(0) = c$ bağıntısını çözün. $T(n) = T(n-2) - c \Rightarrow T(n) = c \left(\frac{n}{2} + 1 \right) \Theta(n)$
- S2.** $T(n) = 3T(n/3) + c$, $T(1) = c$ bağıntısını çözün. ($\Theta = ?$) $T(n) = 3^i T(n/3^i) + c \left(\frac{n-1}{2} \right) \Rightarrow T(n) = c \left(\frac{3n-1}{2} \right) \Theta(n)$
- S3.** Bir fonksiyon $T(n) = 2T(n/2) + n^2$ ise, her seviyedeki toplam iş ne kadar? Büyük- O üst sınırı nedir? _____

S4. Fibonacci hesaplayan naif rekürsif fonksiyon $T(n) = T(n - 1) + T(n - 2) + c$ bağıntısına sahiptir. Bu $\Theta(2^n)$ 'a yakın mıdır? Neden? _____

